SIMATS ENGINEERING
ASSESSMENT TOOL 1

NAME : KAUSHIK NARAYANAN V
REG NO : 192321093
COURSE: CSA1106 – OBJECT ORIENTED ANALYSIS AND DESIGN

Experiment 1: Book Bank Management System

1. Explain the System

The Book Bank Management System is an institutional software solution designed to streamline the equitable distribution of academic textbooks to students for the duration of a semester. Unlike a traditional library, a book bank provides a bundled set of core textbooks. The system initiates when a Book Bank Administrator manually enters a student's details into the interface. This data is transmitted to a Central Computer, which queries the institutional database to verify the student's eligibility (e.g., checking if tuition fees are paid or if there are outstanding fines). If the Central Computer approves the request, the institutional Office physically or digitally issues the required book bundle to the student and updates the inventory records.

2. Identify Functional Requirements

- The system must allow the Book Bank Administrator to input and submit student details (e.g., Student ID, Semester).
- The system must transmit the entered details securely to the Central Computer for verification.
- The Central Computer must cross-reference the student ID against academic and financial records to determine eligibility.
- The system must generate an approval or rejection status based on the verification.
- The system must alert the Office to issue books upon a successful approval.
- The system must update the book inventory once the issue is completed.

3. Identify Non-Functional Requirements

- **Performance:** Eligibility verification should be processed within 3 seconds to avoid queues at the distribution counter.
- **Security:** Student data must be accessed securely, ensuring compliance with academic privacy policies.
- **Reliability:** The Central Computer must be highly available during the peak book distribution weeks at the start of a semester.

4. Identify Actors

- **Book Bank Administrator:** The primary human actor interfacing with the system to initiate the process.
- **Student:** A passive actor (beneficiary) whose details are processed and who receives the books.

- **Central Computer:** An automated system actor responsible for complex business logic and validation.
- **Office / Issuer:** The human or automated actor responsible for the physical issuance of resources.

5. Explain Each Actor

The **Book Bank Administrator** acts as the data entry point; they do not make decisions but merely feed student data into the system. The **Student** is the entity requesting the service; although they might not directly type into the system in this scenario, their physical presence initiates the workflow. The **Central Computer** represents the backend verification subsystem; it holds the authoritative data regarding student standing. The **Office** acts as the fulfillment center, responding to the Central Computer's directives to hand over the physical assets.

6. Identify Use Cases

- Enter Student Details
- Verify Eligibility
- Approve/Reject Request
- Issue Books
- Update Inventory

7. Explain Every Use Case

Enter Student Details: The Administrator inputs the student's academic ID into the UI.
Verify Eligibility: The Central Computer accesses databases to check if the student is entitled to the books (e.g., fees paid, no active bans).
Approve/Reject Request: Based on verification, the Central Computer generates a final decision flag.
Issue Books: Acknowledging the approval flag, the Office physically hands over the books.
Update Inventory: A sub-process triggered after issuance to decrement available stock.

8. Draw a Professional Use Case Diagram

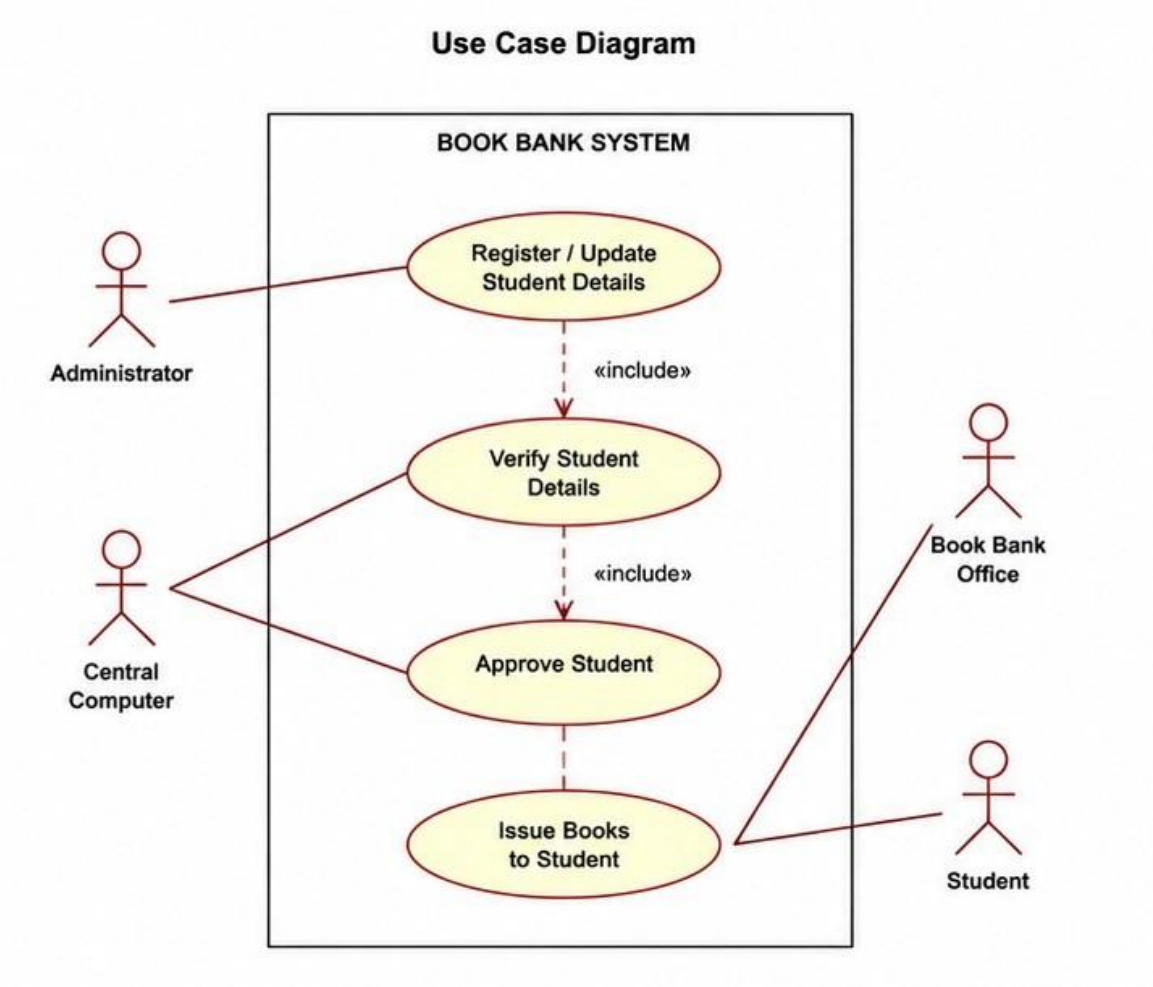


Diagram Explanation: The Administrator triggers *Enter Student Details*. This action strictly requires verification, hence the <<include>> relationship to *Verify Eligibility*, which is processed by the Central Computer system actor. Finally, the Office actor handles *Issue Books*, which inherently includes *Update Inventory* to maintain stock accuracy.

9. Draw a Sequence Diagram

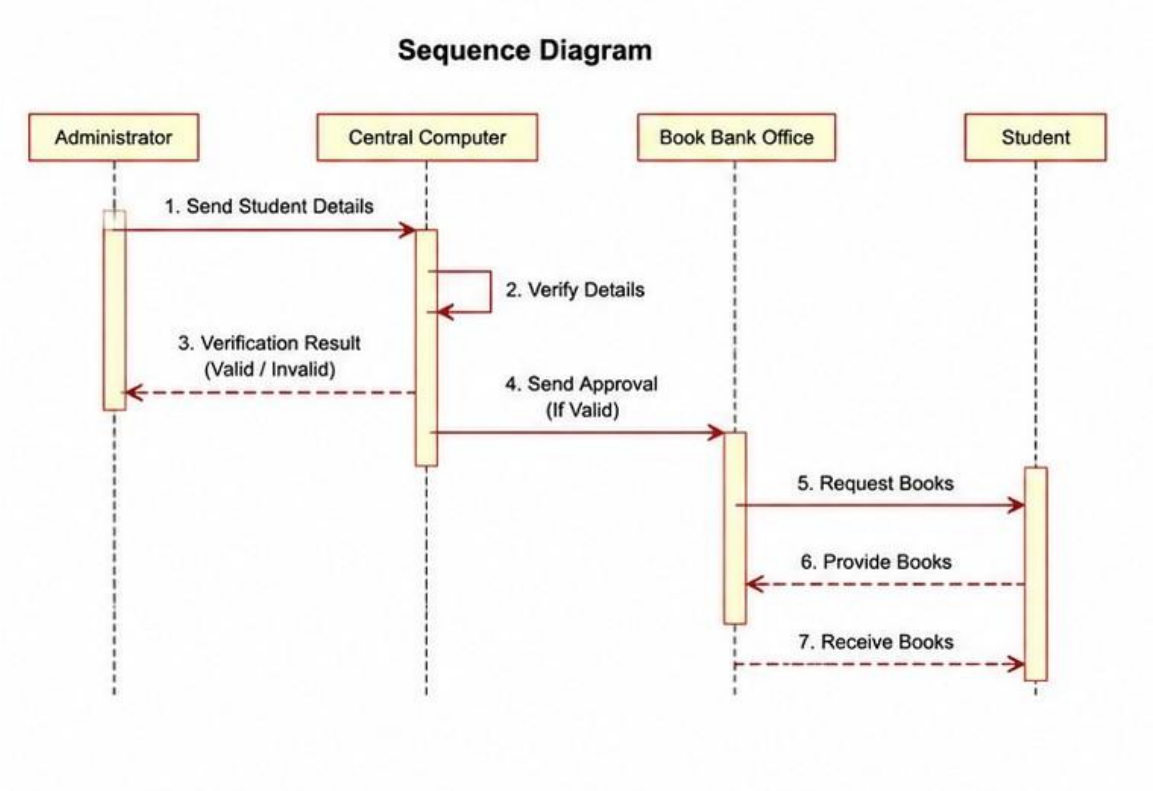
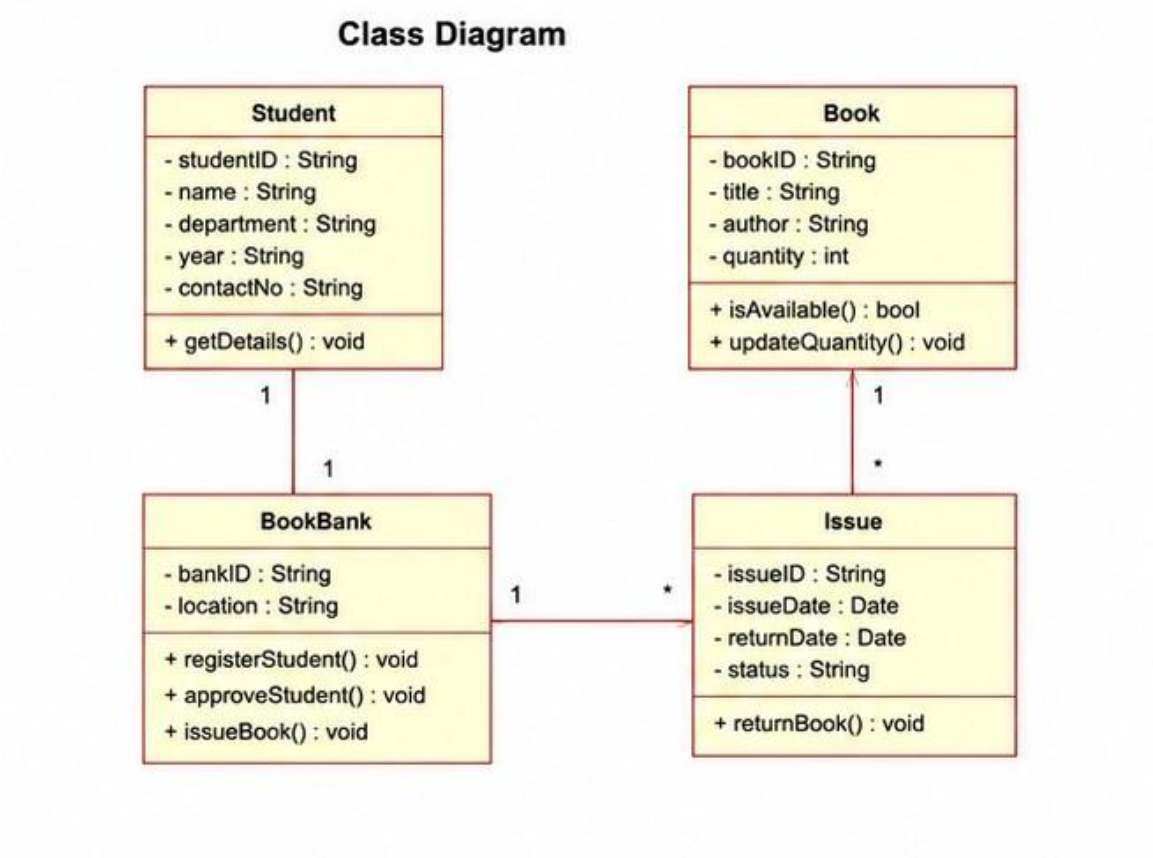


Diagram Explanation: The Sequence Diagram maps the temporal flow. The Administrator inputs data into the UI, which calls the Central Computer. The Central Computer performs a synchronous database query. An **alt** (alternative) fragment is utilized to handle the boolean eligibility status. If true, the system notifies the Office, waits for issuance confirmation, and updates the DB. If false, an error is returned. Activation bars indicate when each object holds the execution thread.

10. Draw a Class Diagram



11. Explain All Relationships

Associations: The Administrator is associated with the Student (1 to 0..*) because one admin can process multiple students. The Central Computer is associated with the Student to perform verification. The Central Computer also has a unidirectional association with the Office, as it sends notifications to the Office upon approval. The Office has an association with the Book class (1 to 1..*) because the office issues one or more books per transaction.
Visibility: Attributes are marked private (-) to enforce encapsulation. Methods are public (+) to define the API interface of each class.

12. Explain the Overall Workflow

The workflow represents a classic three-tier architecture in OOAD. The presentation layer (handled by the Administrator) captures raw data. The business logic layer (Central Computer) performs strict validation against domain rules. Finally, the operational layer (Office) executes the physical transaction resulting from the business logic's decision.

13. Mention Assumptions

- The Book Bank Administrator is already authenticated into the system prior to this workflow.
- The Central Computer has uninterrupted access to the institution's primary fee database.
- All required books for a specific semester are available as a pre-packaged bundle.

14. Write Advantages

Automating the verification process eliminates human error in checking fee receipts. It significantly reduces waiting times for students and ensures real-time accuracy of the book inventory, preventing stockouts during distribution.

15. Conclude the System

The Book Bank Management System effectively digitalizes an administrative bottleneck. Through rigorous Object-Oriented Analysis—mapping out Use Cases, Sequences, and Classes—the architecture ensures secure, reliable, and swift distribution of academic resources, acting as a robust foundation for actual software implementation.

Experiment 2: Exam Registration Portal

1. Explain the System

The Exam Registration Portal is a highly regulated, automated academic system designed to manage candidate enrollments for final examinations. Due to strict institutional compliance, candidates cannot register directly. Instead, their registration details (such as academic history, attendance records, and elected subjects) are submitted to the system through a designated Administrator. This data acts as a formal registration request, which is routed to the Central Computer. The Central Computer operates as the business logic hub, meticulously verifying the candidate's eligibility against rigid institutional criteria (e.g., minimum 75% attendance, cleared tuition dues). Only upon successful systematic approval does the portal authorize the academic Office to physically or digitally issue the official Hall Ticket to the candidate.

2. Requirement Analysis

The requirement analysis phase for this portal focuses on data integrity, workflow authorization, and system reliability. The primary goal is to completely automate the manual verification process traditionally handled by academic clerks, thereby eliminating human error and bias during the crucial pre-examination period.

3. Functional Requirements

- The portal must provide a secure login interface for the Administrator.
- The portal must allow the Administrator to input and submit candidate registration details.
- The Central Computer must fetch candidate attendance and fee records from the central database.
- The Central Computer must evaluate eligibility rules (e.g., attendance >= 75%, no pending fees).
- The portal must notify the Office interface immediately upon candidate approval.
- The portal must allow the Office to generate, print, and issue a unique Hall Ticket.

4. Non-Functional Requirements

- **Auditability:** Every registration request, approval, and rejection must be logged with a timestamp for academic auditing.
- **Scalability:** The portal must handle a surge in concurrent database connections during the final week of the registration deadline.
- **Fault Tolerance:** If the Central Computer loses connection to the database, the transaction must fail safely without issuing a Hall Ticket.

5. Identify Actors

- **Administrator:** A privileged academic staff member acting as the data entry and initiation point.
- **Candidate:** A passive actor representing the student attempting to write the exam.
- **Central Computer:** The core automated backend processing engine and rule evaluator.
- **Office:** The departmental actor responsible for the final artifact (Hall Ticket) generation.

6. Explain Actors

The **Administrator** is a trusted proxy who ensures that the initial data entered into the system is structurally sound. The **Candidate** does not directly interact with the portal's UI in this specific workflow but is the primary subject of the data payload. The **Central Computer** is a critical system actor that holds absolute authority over business logic; it executes the verification algorithms. The **Office** acts as the output mechanism, transforming the Central Computer's digital approval into a tangible or verifiable digital Hall Ticket.

7. Identify Use Cases

- Submit Candidate Details
- Verify Candidate Information
- Approve Registration
- Reject Registration
- Issue Hall Ticket

8. Draw a Use Case Diagram

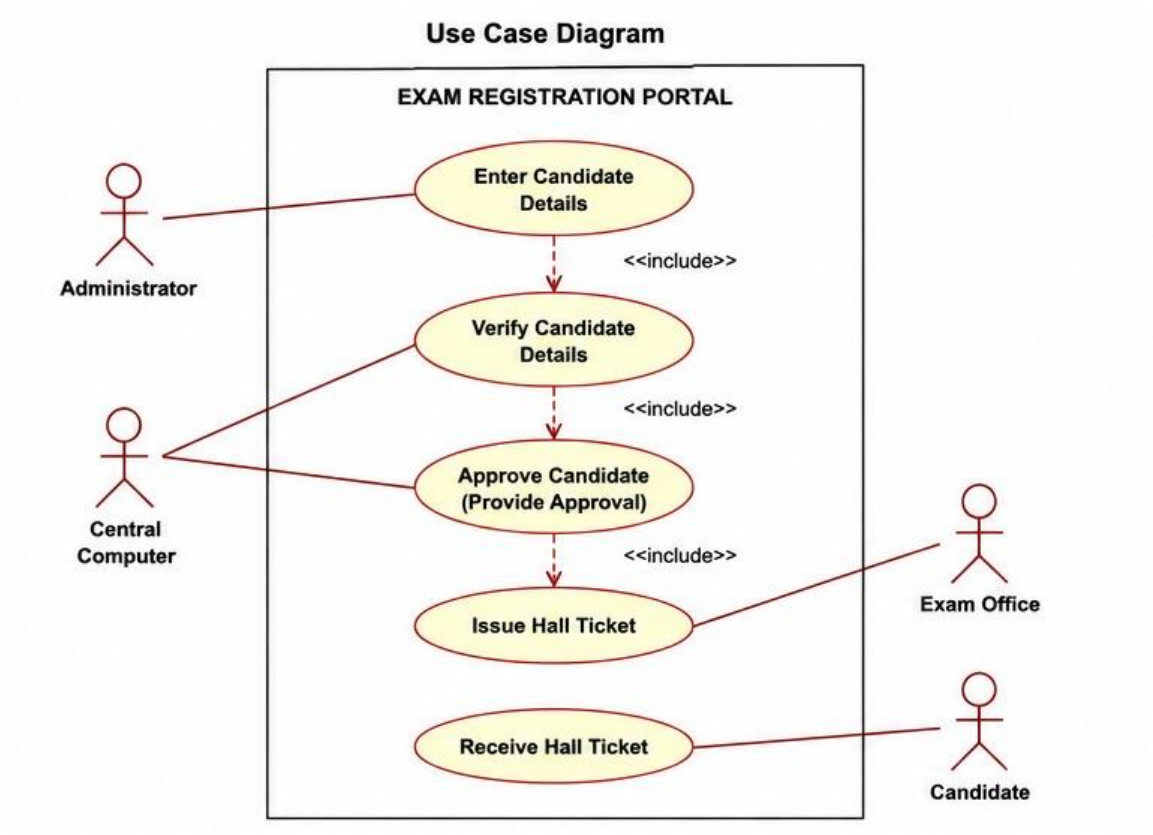


Diagram Explanation: The Administrator triggers *Submit Candidate Details*, which inherently includes (<<include>>) the *Verify Candidate Information* use case executed by the Central Computer. The *Issue Hall Ticket* use case is handled by the Office. Notice the <<extend>> relationship from Verify to Issue: Issuing a Hall Ticket is conditional behavior that only executes *if* the verification is successful.

9. Draw a Sequence Diagram

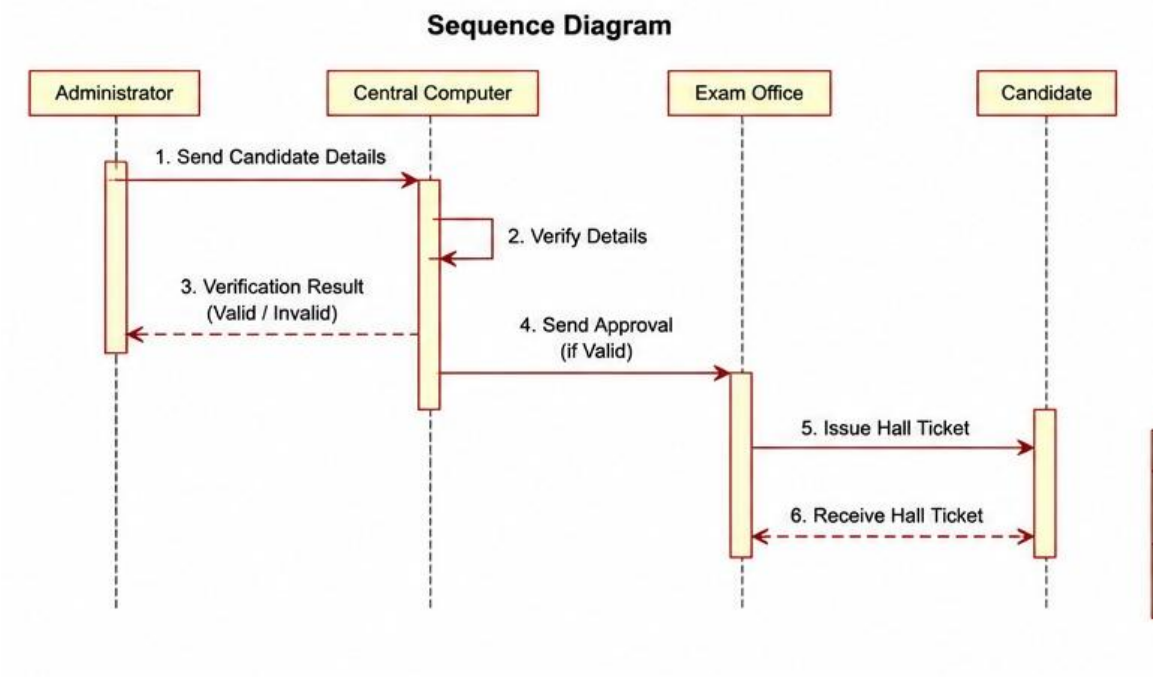
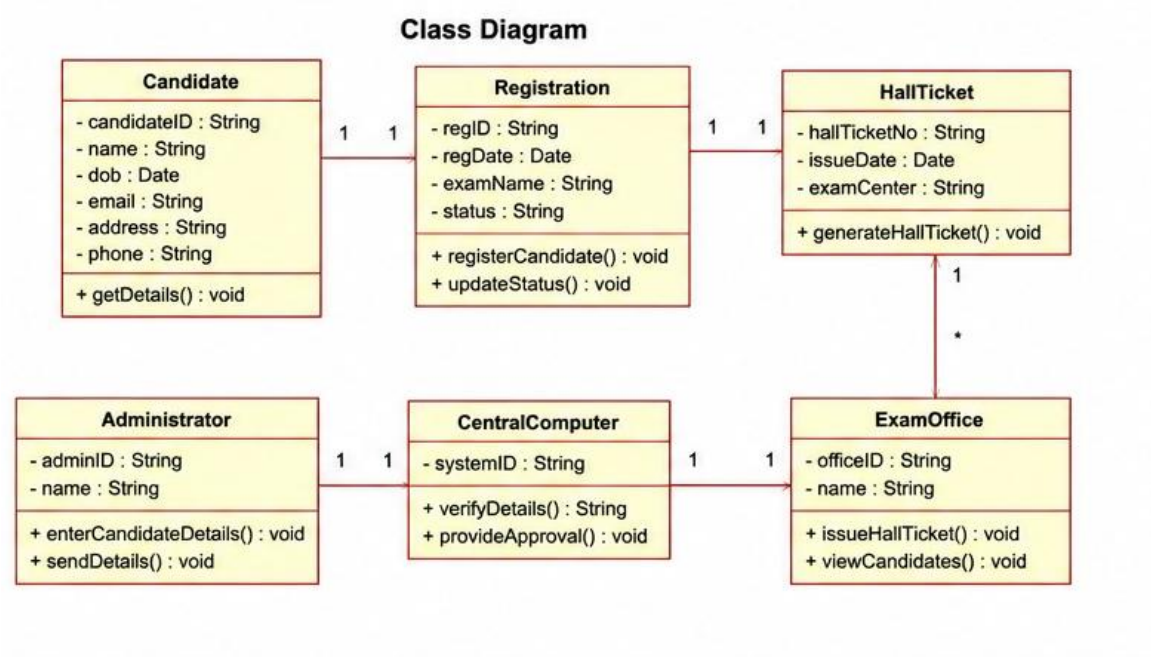


Diagram Explanation: This sequence diagram models the temporal communication flow. The synchronous call submitDetails initiates the sequence. The Central Computer fetches data from the database and performs self-processing (evaluateEligibility). The **alt** block represents the core business logic branch. Upon success, an asynchronous or synchronous message is sent to the Office to generate the ticket. If ineligible, the UI immediately displays a failure reason without involving the Office.

10. Draw a Class Diagram



11. Explain Classes and Relationships

Classes: Administrator and Candidate hold data regarding human entities. CentralComputer handles complex verification methods (checkAttendance, checkFees). Office is the actor proxy class for generating output. HallTicket is the data structure representing the final artifact.

Relationships:

- An Administrator has a one-to-many (1 to 0..*) association with Candidate, submitting multiple registrations.
- The Central Computer verifies Candidates and has a directional association with the Office.
- The Office generates exactly one (1 to 1) Hall Ticket per successful authorization.
- The Hall Ticket holds a direct structural association with a single Candidate.

12. Explain Workflow

The workflow represents a strictly gated, forward-moving pipeline. It shifts the burden of proof from physical paperwork to automated digital querying. The Administrator acts as the gatekeeper against invalid initial submissions. The Central Computer acts as the impartial judge enforcing university regulations. The Office acts as the final executor, turning digital approvals into academic credentials.

13. Mention Assumptions

- The Academic Database is updated in real-time by other institutional systems (e.g., bio-metric attendance systems).

- The Administrator has pre-verified the physical identity of the Candidate before initiating the digital submission.

14. Advantages

This automated portal guarantees absolute adherence to institutional exam policies by removing human subjectivity in eligibility checks. It drastically reduces the administrative workload during exam seasons, prevents unauthorized candidates from receiving hall tickets, and maintains a perfect digital audit trail.

15. Conclusion

In conclusion, the Exam Registration Portal leverages core OOAD principles to modularize an otherwise chaotic administrative process. By segregating data entry (Administrator), business logic (Central Computer), and artifact generation (Office), the system achieves high cohesion and low coupling. The provided UML artifacts accurately capture these requirements, laying the groundwork for a robust software implementation.

Experiment 3: Stock Maintenance System

1. Explain the System

The Stock Maintenance System is a B2B (Business-to-Business) or B2C (Business-to-Consumer) inventory control architecture. It is designed to manage the lifecycle of an order from inception to fulfillment. The workflow triggers when a Customer places an order for specific items. A Stock Dealer (acting as a sales representative or warehouse manager) receives and processes this order. To prevent overselling, the Dealer interfaces with a Central Stock System, which holds the real-time truth of inventory levels. The Central Stock System verifies if the requested quantity is physically available. Upon successful verification, the items are packaged and delivered to the Customer. Crucially, the system then performs a transactional update to deduct the delivered quantity from the active stock, maintaining real-time inventory equilibrium.

2. Requirement Analysis

This system demands high concurrency control and transactional integrity. In an environment where multiple Stock Dealers might be processing orders simultaneously, the Central Stock System must handle data locks to prevent race conditions (e.g., selling the last unit to two different customers). The requirement analysis focuses on real-time availability checks and deterministic inventory synchronization.

3. Functional Requirements

- The system must provide an interface for Customers to place item orders.
- The system must allow the Stock Dealer to receive and process incoming orders.
- The Central Stock System must query the database to verify the availability of requested items against the requested quantity.
- The system must facilitate a delivery workflow upon stock confirmation.
- The Central Stock System must automatically decrement the stock quantity post-delivery or post-confirmation.

4. Non-Functional Requirements

- **Consistency:** The system must adhere to ACID (Atomicity, Consistency, Isolation, Durability) properties during stock updates.
- **Performance:** Stock verification queries must execute in sub-millisecond times to allow rapid order processing.

5. Identify Actors

- **Customer:** The external entity initiating the purchase demand.
- **Stock Dealer:** The internal business user facilitating and managing the order processing.

- **Central Stock System:** The automated backend engine maintaining the source of truth for inventory data.

6. Explain Use Cases

Place Order: The Customer selects items and requests a purchase.
Process Order: The Stock Dealer reviews the order details for structural validity.
Verify Stock Availability: The Central Stock System queries the database to ensure stock_count >= requested_count.
Deliver Items: The physical or logistical process of fulfilling the order.
Update Stock Quantity: The backend transaction that permanently decrements the available stock to reflect the delivery.

7. Draw Use Case Diagram

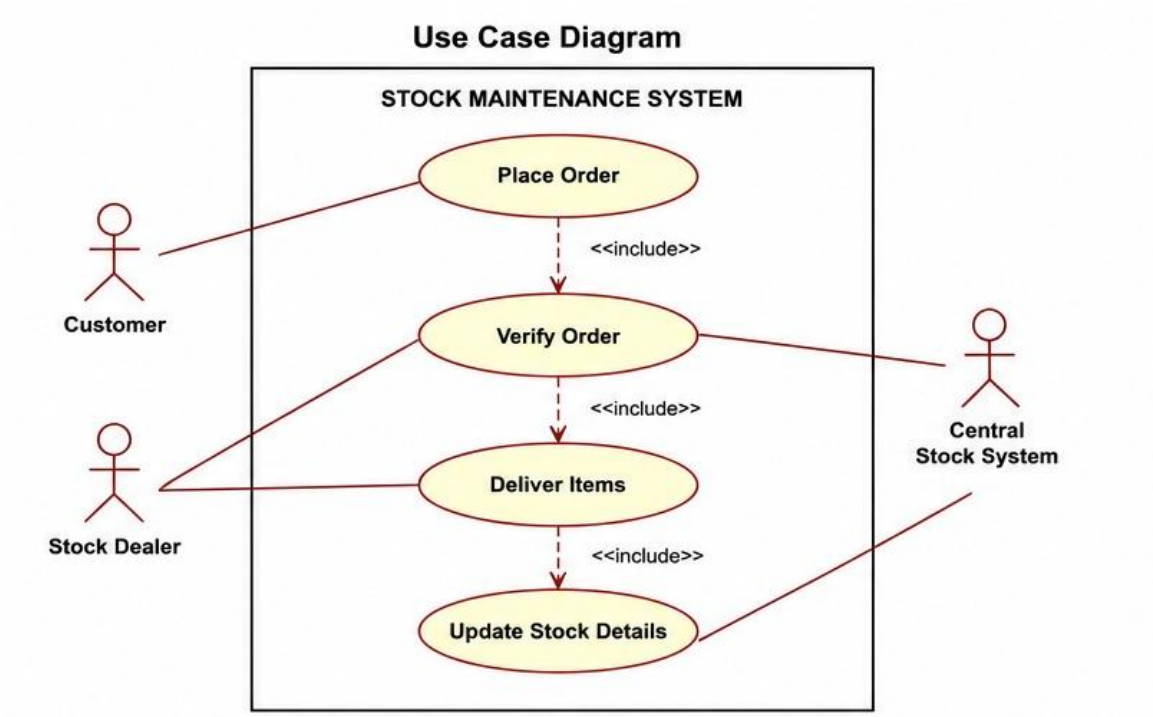


Diagram Explanation: The *Place Order* use case naturally leads to (and strictly requires) the *Process Order* use case (handled by the Dealer). Processing the order includes *Verify Stock Availability* (handled by CSS). Finally, when the Dealer initiates *Deliver Items*, it strictly includes the *Update Stock Quantity* use case to ensure the physical state matches the digital state.

8. Draw Sequence Diagram

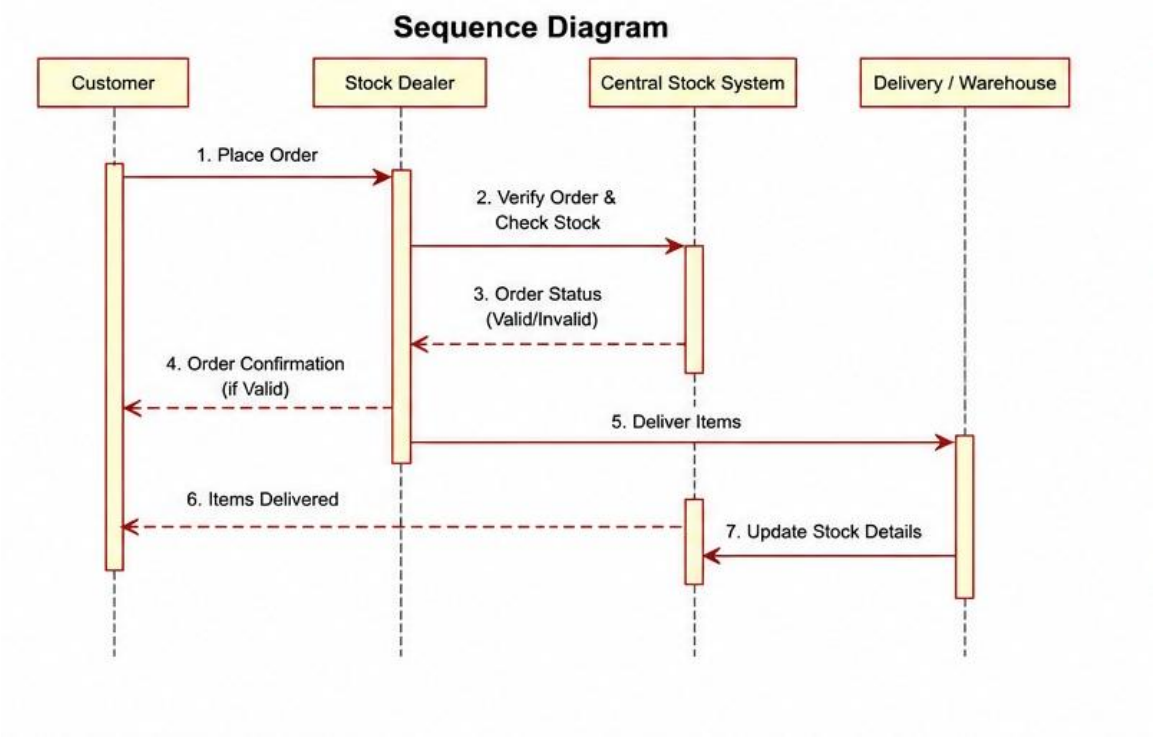
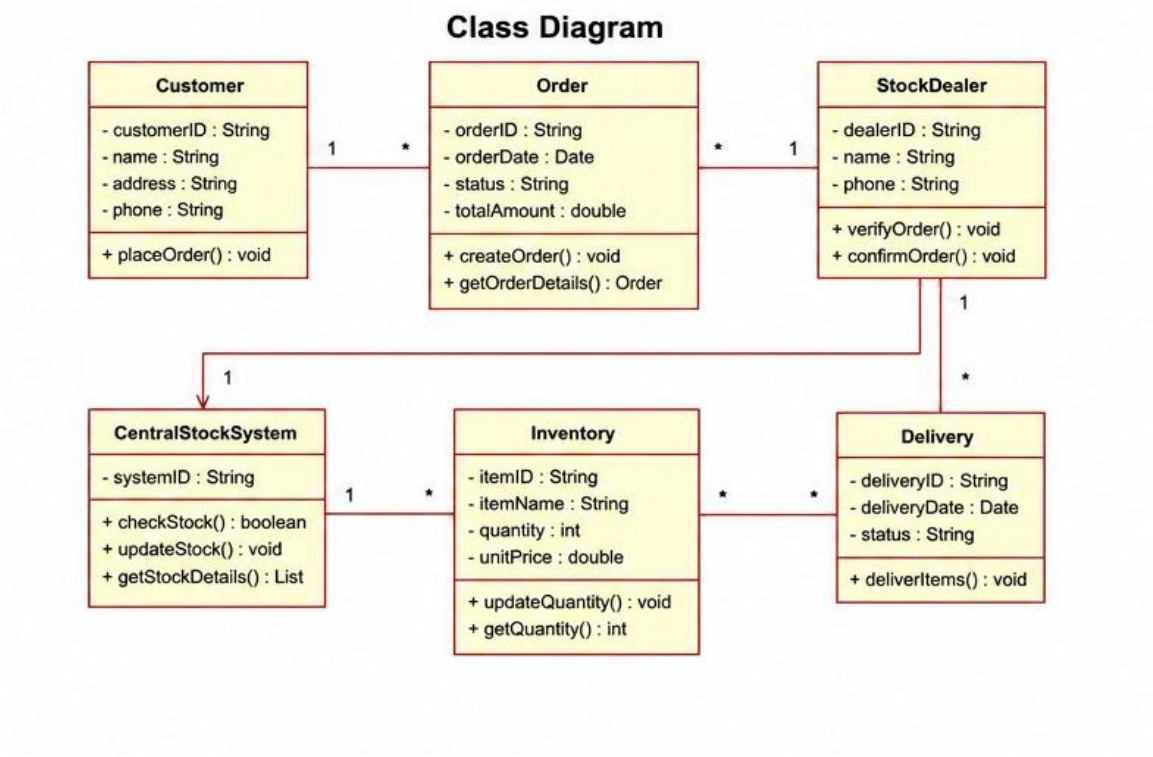


Diagram Explanation: The chronological flow shows the Customer placing an order via the UI, which alerts the Dealer. The Dealer delegates the availability check to the Central Stock System. An **alt** block is used: If sufficient stock exists, the Dealer confirms and delivers, subsequently telling the CSS to finalize the transaction by updating the DB. If stock is insufficient, the system fails fast, and the Dealer rejects the order.

9. Draw Class Diagram



10. Explain Associations

Associations: The Customer places zero or multiple Orders (1 to 0..*). The Stock Dealer processes multiple Orders.
Composition: The Order class has a strong Composition relationship (solid diamond) with the Item class. An Order is fundamentally composed of line items; if the Order is deleted from the system, those specific line item records attached to that order are conceptually destroyed.
Management: The Central Stock System manages the overall pool of Items.

11. Explain Workflow

This workflow highlights synchronous dependency. The Customer acts as the catalyst. The Stock Dealer acts as the orchestrator. However, the orchestrator is powerless without the backend validation of the Central Stock System. Only when physical reality (inventory) aligns with digital requests (orders) does the workflow proceed to physical delivery and final digital reconciliation.

12. Mention Assumptions

- Customers are pre-registered and have a valid delivery address in the system.
- The Stock Dealer has access to physical logistics to fulfill the "Deliver Items" use case.
- Payments are handled out-of-band or via a system not explicitly detailed in this specific inventory-focused scenario.

13. Advantages

The strict separation of concerns prevents inventory drift (where digital stock does not match physical stock). By automating the verification and update processes, the system prevents costly overselling errors, improves customer satisfaction through reliable fulfillment, and provides real-time analytics on stock depletion.

14. Conclusion

In conclusion, the Stock Maintenance System effectively models a transactional e-commerce backend. Through the application of OOAD methodologies, we established a clear boundary between human workflows (Dealer/Customer) and automated data integrity protocols (Central Stock System). The integration of composition relationships in the class diagram and alternative pathways in the sequence diagram creates a highly resilient architectural blueprint.

Experiment 4: Online Course Reservation System

1. Explain the System

The Online Course Reservation System is an academic enrollment platform designed to manage the distribution of limited seats across various university courses. A Student interacts with the platform to browse a catalog of available courses. Upon finding a desired subject, the Student selects the course to initiate the reservation process. The University (represented as a backend systemic entity) intercepts this request to verify seat availability in real-time, as popular courses may fill up concurrently. If seats are available, the system decrements the available seat count and formally confirms the Student's enrollment. If the course is full, the system alerts the student and prevents enrollment, maintaining strict class size limits.

2. Requirement Analysis

The core challenge of this system is managing shared resources (course seats) in a highly concurrent environment (e.g., registration day). The requirement analysis focuses on providing an intuitive browsing experience while enforcing strict, transactional backend validations to prevent over-enrollment.

3. Functional Requirements

- The system must present a searchable, up-to-date catalog of available courses.
- The system must allow a Student to select and request enrollment in a specific course.
- The University system must instantly query the database to check current seat availability.
- The system must securely reserve a seat if available, blocking other concurrent requests for that specific seat.
- The system must generate a confirmed enrollment status for the Student.
- The system must reject the enrollment if the capacity is exactly zero.

4. Non-Functional Requirements

- **Concurrency:** The system must employ database-level locking to handle thousands of simultaneous enrollment requests without overselling seats.
- **Usability:** The browsing interface must be intuitive, displaying real-time seat counts to manage student expectations.

5. Identify Actors

- **Student:** The primary human actor interacting with the frontend to browse and enroll.

- **University (System):** The automated backend actor responsible for managing the business logic, constraints, and data persistence.

6. Explain Actors

The **Student** is the beneficiary and initiator of the workflow. Their actions drive the state changes in the system. The **University** is modeled not as a human administrator, but as the overarching automated system (Server + Database) that enforces academic rules, checks capacities, and executes the formal enrollment transaction.

7. Identify Use Cases

- Browse Courses
- Select Course
- Check Seat Availability
- Confirm Enrollment
- Reject Enrollment

8. Draw Use Case Diagram

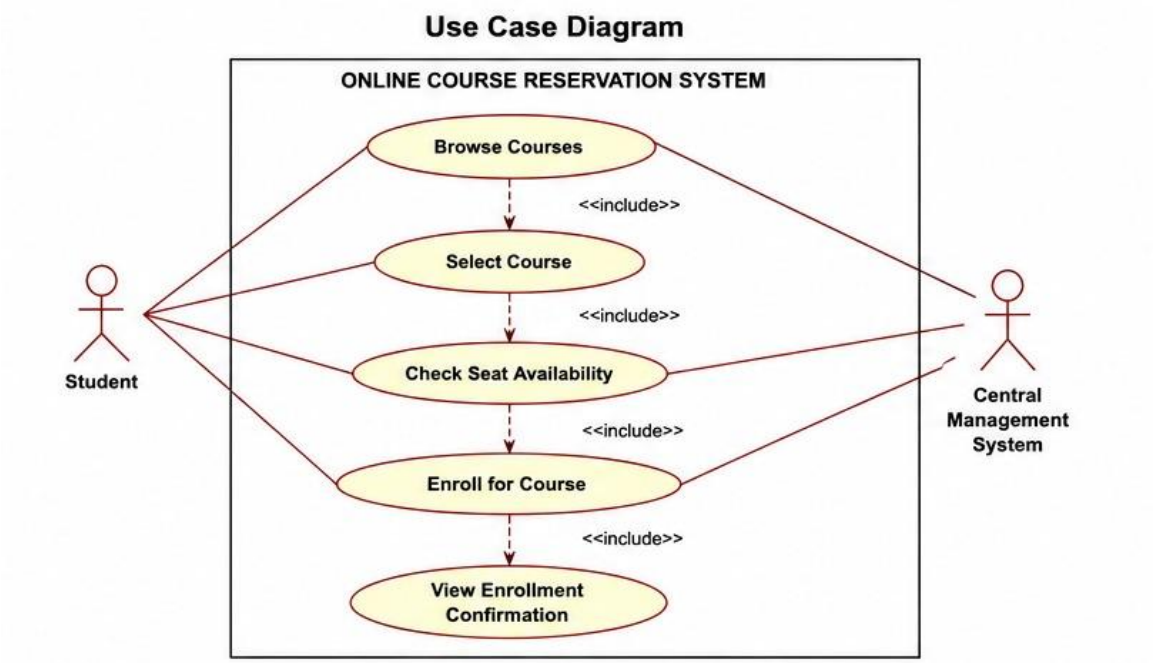


Diagram Explanation: The Student interacts with *Browse Courses* and *Select Course*. Selecting a course inherently requires the system to *Check Seat Availability* (<<include>>). The University system actor executes this check. Based on the result of the check, the process will conditionally extend to either *Confirm Enrollment* (if seats > 0) or *Reject Enrollment* (if seats == 0), modeled using <<extend>> relationships.

9. Draw Sequence Diagram

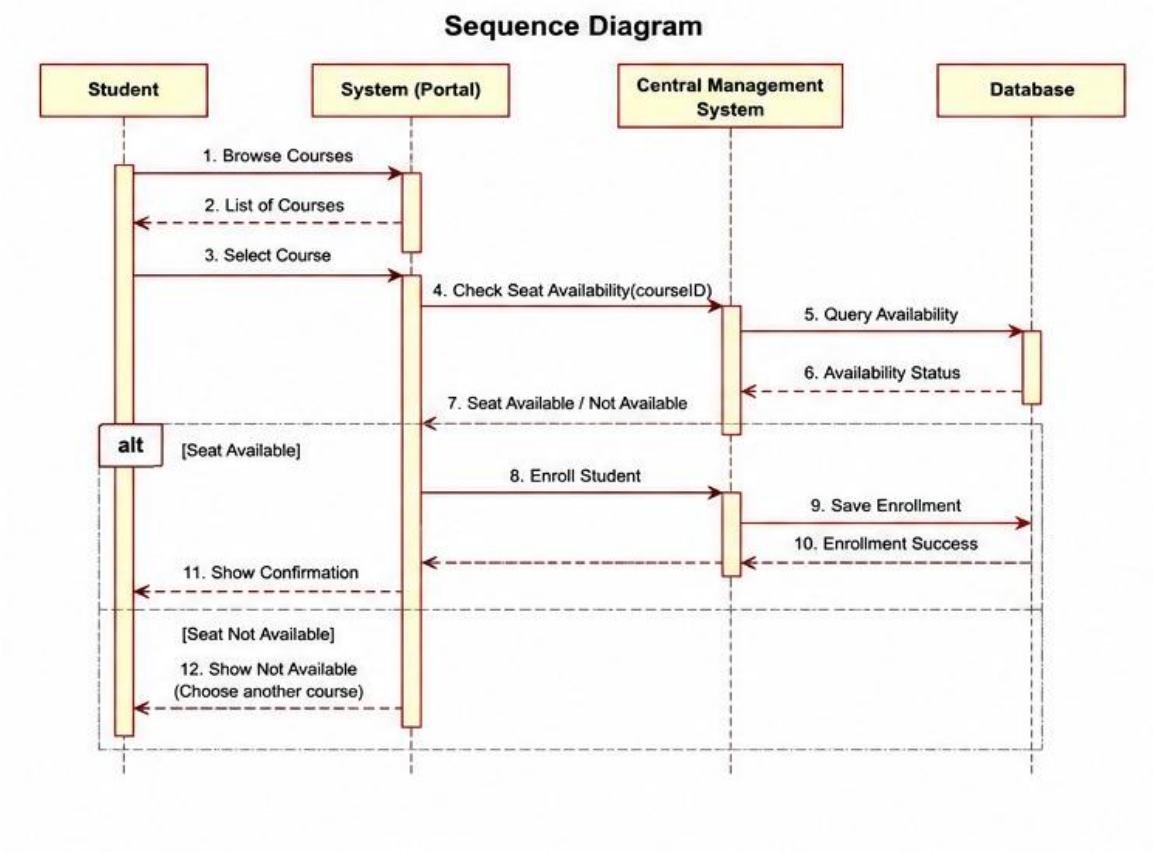
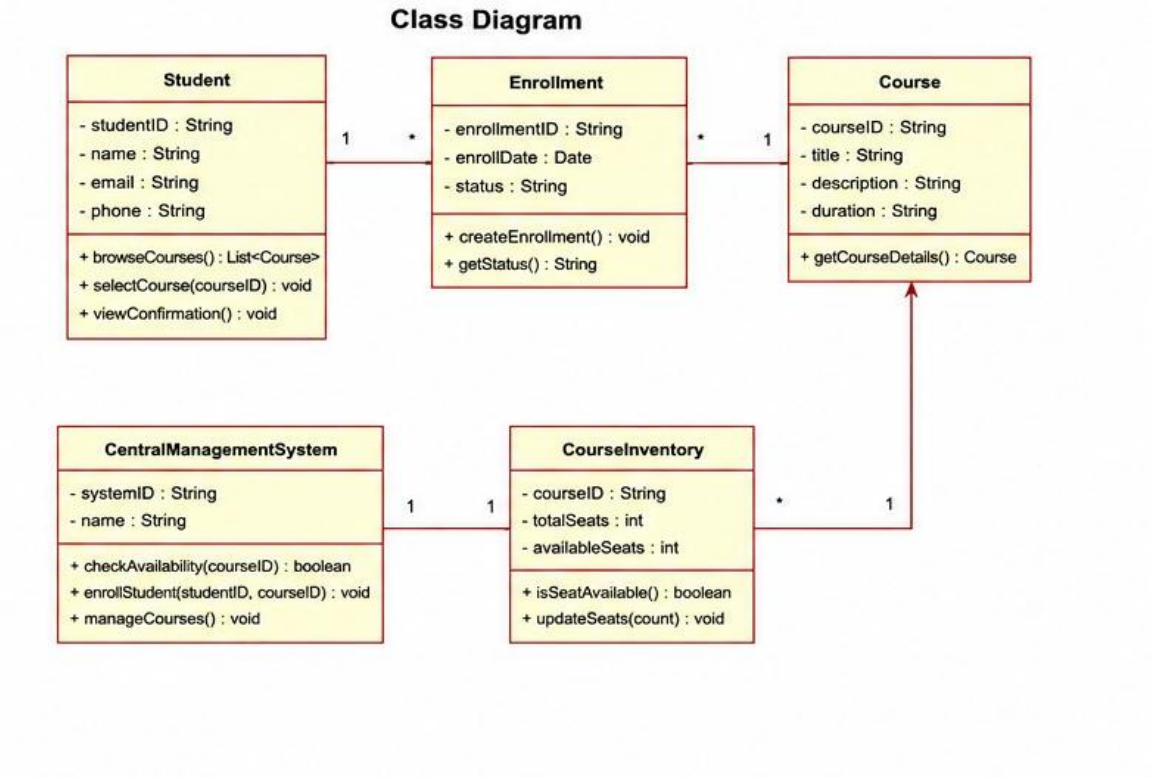


Diagram Explanation: The Sequence Diagram illustrates the interactive loop. The Student browses, then requests a specific course. The Web Portal forwards this to the University backend, which immediately polls the Database for seatCount. The **alt** block defines the divergence: if seats exist, a second DB call is made to persist the assignment (assignSeat), followed by a success cascade back to the Student. If full, a failure cascade immediately returns to the Student.

10. Draw Class Diagram



11. Explain Relationships

Associations: A single Student can make zero or multiple (0..*) Enrollments. The University System manages one to many (1..*) Courses. A single Course can possess zero or multiple (0..*) Enrollments.
Class Structure: The Enrollment class acts as an associative entity (or join table in DB terms) that formally links a Student to a Course. By extracting the transaction into its own class, we achieve high flexibility, allowing us to easily track metadata like enrollment dates and statuses without cluttering the Student or Course classes.

12. Explain Enrollment Workflow

The enrollment workflow is a classic optimistic locking scenario. The user browses stale data (the catalog). When they attempt a write operation (enrollment), the system dynamically fetches the absolute latest state (available seats). Only if the state permits does the system execute the write, immediately decrementing the resource to protect it from subsequent concurrent users.

13. Mention Assumptions

- The Student is successfully authenticated via a Single Sign-On (SSO) or standard login prior to browsing.
- Courses have a hard cap on maxCapacity that cannot be overridden by standard system logic.
- Prerequisite checking (e.g., Student has passed Course A before taking Course B) is handled silently or out-of-scope for this specific capacity-checking scenario.

14. Advantages

The automated reservation system eradicates the chaos of manual registration. It guarantees fairness (first-come, first-served) at the millisecond level, prevents classroom overpopulation which violates safety and academic codes, and provides students with immediate, transparent feedback regarding their academic schedules.

15. Conclusion

In conclusion, the Online Course Reservation System exemplifies the management of constrained resources in software engineering. By employing rigorous OOAD principles—clearly defining the interactive use cases, modeling the temporal data flow via sequence diagrams, and structuring the associative entities in the class diagram—the system ensures data consistency, academic fairness, and operational efficiency.

References

- Object Management Group (OMG), 2017. *OMG Unified Modeling Language (OMG UML) Specification*, Version 2.5.1.
- Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-Oriented Analysis and Design with Applications* (3rd ed.). Addison-Wesley Professional.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley Professional.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3rd ed.). Prentice Hall.
- PlantUML (2024). *PlantUML Language Reference Guide*. Retrieved from https://plantuml.com/